

AI Engineering Task: **Marvel Edition**



Backstory: Project Gene-Forge

In a top-secret initiative code-named Project Gene-Forge, S.H.I.E.L.D. is racing to decode the hidden secrets behind superhuman genetics. Recent discoveries indicate that the powers of Marvel heroes and villains—ranging from Wolverine’s healing factor to Spider-Man’s web-slinging agility—trace back to critical genetic mutations. These revelations have prompted S.H.I.E.L.D. to create a next-generation intelligence system that merges a knowledge graph (mapping out each character’s DNA, affiliations, and powers) with a generative AI model capable of weaving these facts into coherent, detailed answers.

Your mission? Develop this hybrid AI platform so that S.H.I.E.L.D. field operatives can quickly look up any superhuman or supernatural threat—and instantly learn who they are, how they got their powers, and why it matters. By integrating structured genomic data with the creative might of an LLM, you’ll help bridge the gap between cutting-edge science and legendary heroics, ensuring Earth’s mightiest heroes have the intel they need to save the day.

Task Overview

Build a knowledge graph containing Marvel characters, their DNA/genetic mutations, powers, and teams/affiliations. Then use a generative AI model (LLM) that leverages facts from this graph to provide accurate, context-rich answers to user questions.

1. Data & Theme

1. Marvel + DNA Dataset:

- You can create or provide a small, custom dataset (e.g., a CSV or JSON file) that captures:
 - **Character Name** (e.g., “Wolverine”)
 - **Affiliation** (e.g., “X-Men”)
 - **Known Mutations/Genes** (e.g., “Regenerative Mutation Gene X”)
 - **Primary Powers** (e.g., “Accelerated Healing”)
- Include relationships like which gene or mutation is responsible for a given power. For instance, “Gene X” → [CONFERS] → “Accelerated Healing.”

2. Knowledge Graph:

- Each entity (Character, Gene, Power, Team) becomes a node in the graph.
- Relationships:
 - (Character) → [MEMBER_OF] → (Team)
 - (Character) → [HAS_MUTATION] → (Gene)
 - (Gene) → [CONFERS] → (Power)
 - (Character) → [POSSESSES_POWER] → (Power)

3. LLM Text Snippets:

- Provide short textual descriptions about each character’s background (a few sentences).
 - The LLM can use these to generate more context when answering queries.
-

2. Requirements

1. Set Up the Knowledge Graph

- You may use Neo4j or an in-memory graph library (like networkx) for simplicity.
- Insert the nodes and edges from your Marvel+DNA data.

2. Integrate the Graph with an LLM

- On user queries, the system should:
 1. Query the knowledge graph for relevant facts (e.g., which genes a character has).
 2. Pass these facts as context or prompts to a generative language model (an Open AI API Token will be provided).
 3. Return a factual, context-rich answer that references the data.

3. API Endpoints

- **POST /question**: Accepts a user's question (e.g., "*What gene gives Wolverine his healing power?*"), queries the graph for relevant info, and then calls the LLM to construct a response.
 - **GET /graph/{character}**: Returns the character's immediate neighbors in the graph—e.g., all powers, genes, team affiliations, etc.
4. **Optional Enhancements**
- **Caching**: Store repeated queries in Redis or an in-memory cache so you don't always hit the LLM.
 - **Confidence Scores**: If you store confidence levels in the relationships (e.g., some knowledge might be less certain), reflect that in the final response.
 - **Front-End Demo**: Provide a tiny web UI to illustrate queries and responses.
-

3. Deliverables

1. **Graph Schema**
 - Show how you define your entities and relationships. You can provide a diagram or Cypher commands (if using Neo4j).
 2. **Code Repository**
 - Include a straightforward **README** with instructions to run your knowledge graph and AI integration locally or in a Docker container.
 3. **Sample Queries & Responses**
 - Demonstrate a **few example queries** (like "*Which gene is responsible for Spider-Man's powers?*") and **show the LLM's response**.
 - Highlight how the knowledge graph data was used to **ground** or **verify** the answer.
 4. **Brief Explanation**
 - A short write-up describing how you **combined the graph data** with the **LLM**.
 - Clarify any **prompt engineering** or **query logic** you used.
-

****If needed an OpenAPI key will be provided.**